

1 Pemrograman Bash

Bab ini membahas dasar-dasar pemrograman Bash: membuat script, mengelola variabel, menulis struktur kontrol, mendefinisikan fungsi, memproses argumen, dan melakukan debugging. Konsep dibandingkan dengan Java agar mahasiswa yang sudah mengenal pemrograman dapat membangun pemahaman lebih cepat.

Contoh: Bash sebagai bahasa pemrograman

Bayangkan Anda sudah pernah menulis program Java. Java pun memiliki variabel, kondisi, perulangan, dan fungsi. Bash pun demikian, bedanya, Bash menjalankan *perintah sistem operasi* sebagai bagian dari logika programnya. Sebuah script Bash adalah **daftar perintah terminal yang disimpan dalam file**, dilengkapi logika kontrol agar dapat berjalan otomatis tanpa keterlibatan manusia.

1.1 Dasar-dasar Scripting Bash

Cara Membuat dan Mengedit File Script

Bab ini menggunakan dua cara membuat file: **nano** untuk menulis interaktif, dan **heredoc** untuk membuat file langsung dari terminal.

Menggunakan nano:

1. Buka atau buat file: `nano nama-file.sh`
2. Ketik isi file kursor langsung aktif, tidak ada mode khusus.
3. Tombol penting:
 - `Ctrl+O` lalu `Enter` simpan file
 - `Ctrl+X` keluar dari nano
 - `Ctrl+W` cari teks dalam file

Catatan

Di bagian bawah layar nano, selalu tampil daftar shortcut aktif. Tanda ^ berarti Ctrl, jadi ^O = `Ctrl+O`.

Menggunakan **heredoc** (untuk script pendek atau otomatisasi):

```
cat <<'EOF' > nama-file.sh
#!/bin/bash
echo "Halo dari script"
EOF
```

Kode 1.1: Membuat file dengan heredoc

Struktur Dasar dan Shebang Line

Setiap script Bash diawali dengan *shebang line*:

```
#!/bin/bash
# Komentar: jelaskan fungsi script ini

echo "Script berjalan"
```

Kode 1.2: Struktur dasar script Bash

Baris `#!` memberi tahu kernel Linux interpreter mana yang digunakan. Ada dua bentuk yang umum:

- `#!/bin/bash` path absolut ke Bash. Paling umum di Linux, sederhana, dan langsung.
- `#!/usr/bin/env bash` meminta perintah `env` mencari Bash di `$PATH`. Lebih portabel lintas sistem operasi.

Aspek	<code>#!/bin/bash</code>	<code>#!/usr/bin/env bash</code>
Lokasi Bash	Tetap <code>/bin/bash</code>	Dicari di <code>PATH</code>
Portabilitas	Linux standar	Linux dan macOS
Rekomendasi	Praktikum ini	Script lintas platform

Catatan

Shebang hanya bekerja jika file memiliki izin eksekusi (`chmod +x`) dan dijalankan langsung (`./script.sh`). Jika dipanggil `bash script.sh`, shebang diabaikan karena interpreter sudah ditentukan eksplisit.

Setelah membuat script, beri izin eksekusi dan jalankan:

```
chmod +x nama-file.sh
./nama-file.sh
```

Kode 1.3: Memberi izin dan menjalankan script

Praktikum 7.1 Script Pertama: Laporan Sistem

Tujuan: membuat, menyimpan, dan menjalankan script Bash pertama.

1. Buat workspace praktikum:

```
mkdir -p ~/praktikum-os/week09/{scripts,logs,data}
cd ~/praktikum-os/week09/scripts
```

Kode 1.4: Menyiapkan workspace

2. Buat script dengan nano:

```
nano laporan-sistem.sh
```

Kode 1.5: Membuka nano

3. Ketik isi berikut, simpan (**Ctrl+O** **Enter**), lalu keluar (**Ctrl+X**):

```
#!/bin/bash
# Script: laporan-sistem.sh

echo "===== "
echo "  LAPORAN SISTEM"
echo "===== "
echo "Tanggal   : $(date '+%A, %d %B %Y') "
echo "Jam       : $(date '+%H:%M:%S') "
echo "Hostname  : $(hostname) "
echo "User      : $(whoami) "
echo "CPU core  : $(nproc) "
echo "RAM bebas: $(free -h | awk '/^Mem/ {print $4}')"
echo "Disk /    : $(df -h / | awk 'NR==2 {print $5}')"
echo "    terpakai"
echo "===== "
```

Kode 1.6: Isi laporan-sistem.sh

4. Beri izin dan jalankan:

```
chmod +x laporan-sistem.sh
./laporan-sistem.sh
```

Kode 1.7: Menjalankan script

Latihan Latihan 9.1

Modifikasi `laporan-sistem.sh` agar menyimpan output ke file `laporan-YYYY-MM-DD.txt` sekaligus menampilkannya di terminal. Petunjuk: gunakan `tee` yang sudah dipelajari di bab sebelumnya.

1.2 Variabel dan Parameter Posisional

Variabel di Bash vs Java

Di Java: `String nama = "Budi";`. Di Bash, aturannya berbeda:

- **Deklarasi:** `NAMA="Budi"` tanpa spasi di sekitar `=`
- **Akses nilai:** `$NAMA` atau `${NAMA}`
- **Tipe data:** tidak ada. Semua variabel Bash adalah string secara default; aritmetika memerlukan sintaks khusus.

Peringatan

`NAMA = "Budi"` (dengan spasi) menghasilkan error Bash mengira `NAMA` adalah nama perintah. Selalu tulis tanpa spasi.

Variabel khusus yang paling sering dipakai dalam scripting:

Variabel	Nilai
<code>\$0</code>	Nama script yang sedang berjalan
<code>\$1, \$2, ...</code>	Argumen ke-1, ke-2, dst.
<code>\$#</code>	Jumlah argumen yang diberikan
<code>@</code>	Semua argumen sebagai daftar terpisah
<code>?</code>	Exit code perintah terakhir (0 = sukses)
<code>\$\$</code>	PID script saat ini

Parameter Posisional dan Nilai Default

Seperti metode Java yang menerima argumen (`void fungsi(String a, String b)`), script Bash menerima argumen dari command line. Argumen dibaca dengan `$1`, `$2`, dst.

Konstruksi `${1:-"default"}` membaca `$1`; jika kosong, gunakan nilai default. Setara dengan `args.length > 0 ? args[0] : "default"` di Java.

Aritmetika dan Substitusi Perintah

Bash tidak langsung menghitung ekspresi aritmetika. Gunakan `$(( ))`:

```
A=10
B=3

echo "Jumlah : $((A + B))"
echo "Modulo : $((A % B))"
echo "Pembagian: $((A / B))"    # Hasil bulat, sama seperti
                                int di Java

# Substitusi perintah: simpan output ke variabel
TANGGAL=$(date '+%F')
JUMLAH_FILE=$(ls | wc -l)
echo "Tanggal: $TANGGAL, File: $JUMLAH_FILE"
```

Kode 1.8: Aritmetika dan substitusi perintah

Catatan

Di Java, `10 / 3` untuk tipe `int` menghasilkan 3. Di Bash, `$((10 / 3))` juga menghasilkan 3. Gunakan `bc` jika butuh presisi desimal.

Praktikum 7.2 Script Info Sistem dengan Argumen

Tujuan: berlatih variabel, substitusi perintah, parameter posisional, dan nilai default.

1. Buat script:

```
nano ~/praktikum-os/week09/scripts/info-sistem.sh
```

Kode 1.9: Membuat script info-sistem.sh

1.2 Variabel dan Parameter Posisional

2. Ketik isi berikut:

```
#!/bin/bash
# Penggunaan: ./info-sistem.sh [nama-admin] [batas-disk-persen]

ADMIN=${1:-"Tidak dikenal"}
BATAS=${2:-80}
TANGGAL=$(date '+%F %T')
DISK_PERSEN=$(df / | awk 'NR==2 {print $5}' | tr -d '%')

echo "Admin      : $ADMIN"
echo "Tanggal    : $TANGGAL"
echo "Disk /      : ${DISK_PERSEN}% terpakai"
echo "Batas       : ${BATAS}%"

if [ "$DISK_PERSEN" -gt "$BATAS" ]; then
    echo "STATUS      : PERINGATAN - disk melebihi batas!"
else
    SISA=$((BATAS - DISK_PERSEN))
    echo "STATUS      : Normal (sisanya toleransi ${SISA}%)"
fi
```

Kode 1.10: Isi info-sistem.sh

3. Simpan, beri izin, uji dengan berbagai kombinasi argumen:

```
chmod +x ~/praktikum-os/week09/scripts/info-sistem.sh
./info-sistem.sh
./info-sistem.sh "Dian" 50
./info-sistem.sh "Dian" 10
```

Kode 1.11: Menguji script

Latihan Latihan 9.2

Buat script `kalkulator.sh` yang menerima tiga argumen: `<angka1>` `<operator>` `<angka2>` dengan operator `+`, `-`, `*`, atau `/`. Contoh: `./kalkulator.sh 20 + 5` menghasilkan 25. Gunakan `case` untuk memilih operasi, dan validasi jika argumen tidak lengkap.

1.3 Struktur Kontrol

Struktur kontrol di Bash konsepnya sama dengan Java, hanya sintaksnya berbeda.

Konsep	Java	Bash
Kondisi if	if (x > 5) {	if [\$x -gt 5]; then
Blok kode	kurung kurawal	then ... fi
Else if	else if (...) {	elif ...; then
Tidak sama	!=	!= (string), -ne (angka)
Dan / Atau	&& /	&& /
Switch	switch (x) { case "a": }	case \$x in "a") ... ;;

Operator Perbandingan

Angka: -eq, -ne, -gt, -lt, -ge, -le

String: = atau == (sama), != (tidak sama), -z (kosong), -n (tidak kosong)

File: -f (file ada), -d (direktori ada), -r (bisa dibaca), -x (bisa dieksekusi)

Perulangan for dan while

```
# for iterasi atas daftar (mirip for-each Java)
for BULAN in Januari Februari Maret; do
    echo "Bulan: $BULAN"
done

# for C-style (mirip Java)
for ((I=1; I<=5; I++)); do
    echo "Angka: $I"
done

# for atas file
for FILE in ~/praktikum-os/week09/scripts/*.sh; do
    echo "Script: $(basename $FILE)"
done

# while selama kondisi benar
COUNTER=1
while [ $COUNTER -le 3 ]; do
    echo "Iterasi $COUNTER"
```

```
COUNTER=$((COUNTER + 1))  
done
```

Kode 1.12: Perulangan for dan while

Struktur case

case setara switch di Java. Setiap blok diakhiri ;; (bukan break). Pola *) setara default:.

Catatan

Di Java, lupa menulis break menyebabkan *fall-through*. Di Bash, ;; sudah otomatis menghentikan eksekusi blok tidak ada fall-through secara default.

Praktikum 7.3 Script Grading dan Menu Interaktif

Tujuan: menggabungkan if, for, while, dan case dalam satu session.

1. Buat script grading (menggunakan if dan for):

```
nano ~/praktikum-os/week09/scripts/grading-batch.sh
```

Kode 1.13: Membuat script grading-batch.sh

2. Ketik isi berikut:

```
#!/bin/bash  
# Script: grading-batch.sh  
# Proses daftar nilai mahasiswa  
  
MAHASISWA=("Andi:92" "Budi:73" "Citra:55" "Deni:80" "  
Eka:45")  
  
echo "=== HASIL GRADING ==="  
for ENTRI in "${MAHASISWA[@]}"; do  
    NAMA=$(echo "$ENTRI" | cut -d: -f1)  
    NILAI=$(echo "$ENTRI" | cut -d: -f2)  
  
    if [ "$NILAI" -ge 85 ]; then  
        GRADE="A"  
    elif [ "$NILAI" -ge 75 ]; then
```



```

        GRADE="B"
    elif [ "$NILAI" -ge 65 ]; then
        GRADE="C"
    elif [ "$NILAI" -ge 55 ]; then
        GRADE="D"
    else
        GRADE="E"
    fi

    printf "%-10s %3d  %s\n" "$NAMA" "$NILAI" "$GRADE"
done
echo "===== "

```

Kode 1.14: Isi grading-batch.sh

3. Simpan, beri izin, dan jalankan:

```

chmod +x ~/praktikum-os/week09/scripts/grading-batch.
sh
./grading-batch.sh

```

Kode 1.15: Menjalankan script grading

4. Buat script menu interaktif (while + case):

```

nano ~/praktikum-os/week09/scripts/menu-sistem.sh

```

Kode 1.16: Membuat script menu-sistem.sh

5. Ketik isi berikut:

```

#!/bin/bash
# Menu interaktif pemantauan sistem

while true; do
    echo ""
    echo "===== MENU MONITOR ====="
    echo "1) Info disk"
    echo "2) Info memori"
    echo "3) Proses teratas"
    echo "4) Keluar"
    echo -n "Pilih [1-4]: "
    read PILIHAN

```

```
case $PILIHAN in
    1) df -h ;;
    2) free -h ;;
    3) ps aux --sort=-%cpu | head -6 ;;
    4) echo "Sampai jumpa!"; exit 0 ;;
    *) echo "Pilihan tidak valid." ;;
esac
done
```

Kode 1.17: Isi menu-sistem.sh

6. Beri izin dan jalankan, coba setiap opsi:

```
chmod +x ~/praktikum-os/week09/scripts/menu-sistem.sh
./menu-sistem.sh
```

Kode 1.18: Menjalankan menu

Latihan Latihan 9.3

Tambahkan ke script `grading-batch.sh` sebuah ringkasan di bagian bawah yang menampilkan: jumlah mahasiswa per grade (A, B, C, D, E) menggunakan perulangan `for` kedua yang mengiterasi array `MAHASISWA`.

1.4 Fungsi dalam Shell Script

Fungsi di Bash vs Java

Di Java: `void namaMetode(String p1, String p2) { ... }`. Di Bash, fungsi didefinisikan tanpa daftar parameter eksplisit argumen dikirim seperti command line dan dibaca dengan `$1`, `$2`, dst.

Konsep	Java	Bash
Definisi	<code>void f(String a, int b)</code>	<code>f() { ... }</code>
Panggil	<code>f("x", 2)</code>	<code>f x 2</code>
Argumen	<code>a, b</code>	<code>\$1, \$2</code>
Return nilai	<code>return nilai</code>	<code>echo nilai (via stdout)</code>
Return status	<code>exception</code>	<code>return 0-255</code>
Variabel lokal	<code>default lokal</code>	wajib tulis <code>local</code>

Catatan

Di Bash, `return` hanya mengembalikan kode status (0–255). Untuk mengembalikan nilai seperti string, gunakan `echo` di dalam fungsi lalu tangkap dengan substitusi: `HASIL=$(nama_fungsi arg)`. Di Java, variabel dalam metode otomatis lokal. Di Bash, tanpa `local`, variabel bocor ke lingkup global.

```
#!/bin/bash

# Fungsi mengembalikan nilai via stdout
hitung_luas() {
    local PANJANG=$1
    local LEBAR=$2
    echo $((PANJANG * LEBAR))
}

# Tangkap nilai dengan substitusi perintah
LUAS=$(hitung_luas 8 5)
echo "Luas: $LUAS cm2"

# Fungsi mengembalikan status sukses/gagal
bagi() {
    local A=$1 B=$2
    if [ "$B" -eq 0 ]; then
        echo "Error: pembagi nol" >&2
        return 1
    fi
    echo $((A / B))
}

HASIL=$(bagi 20 4) && echo "Hasil: $HASIL"
bagi 10 0 || echo "Exit code: $?"
```

Kode 1.19: Mendefinisikan dan memanggil fungsi

Praktikum 7.4 Library Fungsi Validasi

Tujuan: membuat file library fungsi yang dapat dimuat (`source`) oleh script lain konsep serupa dengan *import* di Java.

1. Buat file library:

```
nano ~/praktikum-os/week09/scripts/lib-validasi.sh
```

Kode 1.20: Membuat lib-validasi.sh

2. Ketik isi berikut:

```
#!/bin/bash
# lib-validasi.sh - Library fungsi validasi

adalah_angka() {
    [[ "$1" =~ ^[0-9]+$ ]]
}

file_bisa_dibaca() {
    [ -f "$1" ] && [ -r "$1" ]
}

error_exit() {
    echo "ERROR: $1" >&2
    exit 1
}

info() { echo "[INFO] $1"; }
sukses() { echo "[OK] $1"; }
```

Kode 1.21: Isi lib-validasi.sh

3. Buat script yang menggunakan library:

```
nano ~/praktikum-os/week09/scripts/pakai-library.sh
```

Kode 1.22: Membuat script pakai-library.sh

4. Ketik isi berikut:

```
#!/bin/bash
# Muat library (seperti import di Java)
source ~/praktikum-os/week09/scripts/lib-validasi.sh

ANGKA=$1
FILE=$2

[ -z "$ANGKA" ] || [ -z "$FILE" ] && \
```

```

        error_exit "Penggunaan: $0 <angka> <path-file>"

    if adalah_angka "$ANGKA"; then
        sukses "Input '$ANGKA' adalah angka valid"
    else
        error_exit "'$ANGKA' bukan angka"
    fi

    if file_bisa_dibaca "$FILE"; then
        sukses "File '$FILE' bisa dibaca"
        info "Jumlah baris: $(wc -l < "$FILE")"
    else
        error_exit "File '$FILE' tidak ada atau tidak bisa
        dibaca"
    fi

```

Kode 1.23: Isi pakai-library.sh

5. Beri izin dan uji semua skenario:

```

chmod +x ~/praktikum-os/week09/scripts/pakai-library.
sh
./pakai-library.sh 42 /etc/hostname
./pakai-library.sh abc /etc/hostname
./pakai-library.sh 42 /tidak-ada.txt
./pakai-library.sh

```

Kode 1.24: Menguji semua skenario

Latihan Latihan 9.4

Tambahkan fungsi konfirmasi() ke lib-validasi.sh. Fungsi ini menampilkan pertanyaan, membaca input Y/N dari user, mengembalikan 0 jika Y dan 1 jika N. Buat script demo yang memanggil fungsi ini sebelum menghapus sebuah file.

1.5 Pengolahan Argumen Command Line

Script yang baik memproses argumen secara fleksibel: validasi jumlah argumen, dan opsi seperti `-v` atau `-o file`.

`getopts` adalah cara standar Bash memproses opsi mirip cara kerja perintah Unix seperti `ls -la` atau `grep -i`:

```
#!/bin/bash
VERBOSE=false
OUTPUT_FILE=""

while getopts "vo:" OPSI; do
    case $OPSI in
        v) VERBOSE=true ;;
        o) OUTPUT_FILE="$OPTARG" ;;
        *) echo "Penggunaan: $0 [-v] [-o file]"; exit 1 ;;
    esac
done
shift $((OPTIND - 1)) # Geser argumen agar $1 = argumen
                     non-opsi pertama

echo "Verbose : $VERBOSE"
echo "Output  : ${OUTPUT_FILE:-tidak ada}"
echo "Sisa    : $@"
```

Kode 1.25: Pola dasar `getopts`

Dalam `"vo:"`, tanda `:` setelah huruf berarti opsi itu memerlukan nilai (`-o nama-file`), bukan sekadar flag (`-v`).

Praktikum 7.5 Script Backup dengan Opsi

Tujuan: membuat script yang memproses opsi `getopts` dan mengintegrasikan semua konsep sebelumnya.

1. Buat script:

```
nano ~/praktikum-os/week09/scripts/backup-data.sh
```

Kode 1.26: Membuat `backup-data.sh`

2. Ketik isi berikut:

```
#!/bin/bash
# Penggunaan: ./backup-data.sh [-v] [-c] [-l logfile]
# <sumber> <tujuan>
```

```

VERBOSE=false
COMPRESS=false
LOG_FILE=""

while getopts "vcl:" OPSI; do
    case $OPSI in
        v) VERBOSE=true ;;
        c) COMPRESS=true ;;
        l) LOG_FILE="$OPTARG" ;;
        *) echo "Penggunaan: $0 [-v] [-c] [-l logfile]
            <sumber> <tujuan>"
            exit 1 ;;
    esac
done
shift $((OPTIND - 1))

SUMBER=$1
TUJUAN=$2

log() {
    local MSG="[$(date '+%T')] $1"
    echo "$MSG"
    [ -n "$LOG_FILE" ] && echo "$MSG" >> "$LOG_FILE"
}

[ -z "$SUMBER" ] || [ -z "$TUJUAN" ] && {
    echo "ERROR: sumber dan tujuan wajib diisi"; exit
    1; }

[ ! -d "$SUMBER" ] && { log "ERROR: $SUMBER tidak ada"
    ; exit 2; }

mkdir -p "$TUJUAN"
TANGGAL=$(date '+%F-%H%M%S')
[ "$VERBOSE" = true ] && log "Sumber: $SUMBER | Tujuan
    : $TUJUAN"

if [ "$COMPRESS" = true ]; then
    ARSIP="$TUJUAN/backup-$(basename "$SUMBER")-
    $TANGGAL.tar.gz"
    tar -czf "$ARSIP" -C "$(dirname "$SUMBER")" "$(

```

```

    basename "$SUMBER")"
    log "Arsip: $ARSIP ($(du -sh "$ARSIP" | cut -f1))"
else
    cp -r "$SUMBER" "$TUJUAN/backup-$(basename "$
$SUMBER")-$TANGGAL"
    log "Backup selesai."
fi

```

Kode 1.27: Isi backup-data.sh

3. Beri izin dan uji:

```

chmod +x ~/praktikum-os/week09/scripts/backup-data.sh
cd ~/praktikum-os/week09/scripts

# Tanpa opsi
./backup-data.sh ~/praktikum-os/week09/data ~/
praktikum-os/week09/logs

# Dengan verbose dan kompresi + log ke file
./backup-data.sh -v -c -l ../logs/backup.log \
~/praktikum-os/week09/data ~/praktikum-os/week09/
logs

cat ../logs/backup.log

```

Kode 1.28: Menguji script backup

1.6 Debugging Shell Script

Debugging Bash berbeda dari Java yang memiliki debugger interaktif. Bash menyediakan mekanisme tracing untuk memperlihatkan setiap baris yang dieksekusi.

Flag	Arti	Cara aktifkan
-x	Tampilkan setiap perintah sebelum dijalankan	<code>bash -x script.sh</code>
-e	Keluar jika ada perintah gagal	<code>set -e</code>
-u	Keluar jika variabel tidak terdefinisi	<code>set -u</code>
-n	Cek sintaks tanpa menjalankan	<code>bash -n script.sh</code>

Kombinasi `set -euo pipefail` di awal script menangkap banyak kesalahan umum secara otomatis. Tambahkan `trap` untuk cleanup:


```
#!/bin/bash
set -euo pipefail

cleanup() { echo "[CLEANUP] selesai."; }
trap cleanup EXIT      # cleanup dipanggil selalu saat
                        script berakhir

DEBUG=${DEBUG:-false}
debug() { [ "$DEBUG" = true ] && echo "[DEBUG] $*" >&2; }

debug "Script dimulai dengan PID $$"
echo "Script berjalan normal"
```

Kode 1.29: Pola script defensif

Catatan

trap cleanup EXIT setara dengan finally di Java memastikan cleanup dijalankan meskipun script gagal di tengah jalan.

Praktikum 7.6 Debugging Script

Tujuan: menggunakan teknik debugging untuk menganalisis dan memperbaiki script.

1. Buat script untuk dianalisis:

```
nano ~/praktikum-os/week09/scripts/debug-latihan.sh
```

Kode 1.30: Membuat debug-latihan.sh

2. Ketik isi berikut:

```
#!/bin/bash
# Script: debug-latihan.sh
# Penggunaan: ./debug-latihan.sh <direktori> <batas-MB>
>

DIREKTORI=$1
BATAS=$2

if [ $# -ne 2 ]; then
    echo "Penggunaan: $0 <direktori> <batas-MB>"
```

```
        exit 1
    fi

    UKURAN=$(du -sm "$DIREKTORI" | cut -f1)

    echo "Direktori : $DIREKTORI"
    echo "Ukuran    : ${UKURAN} MB"
    echo "Batas      : ${BATAS} MB"

    if [ "$UKURAN" -gt "$BATAS" ]; then
        echo "PERINGATAN: Ukuran melebihi batas!"
        echo "Kelebihan: $((UKURAN - BATAS)) MB"
    else
        echo "Status: Normal (sisanya: $((BATAS - UKURAN)) MB)"
    fi
fi
```

Kode 1.31: Isi debug-latihan.sh

3. Cek sintaks, lalu jalankan dengan tracing:

```
chmod +x ~/praktikum-os/week09/scripts/debug-latihan.sh
bash -n debug-latihan.sh && echo "Sintaks OK"
bash -x debug-latihan.sh /etc 10
./debug-latihan.sh /var 50
./debug-latihan.sh
```

Kode 1.32: Menganalisis script dengan debugging

Latihan Latihan 9.5

Script `debug-latihan.sh` tidak menangani direktori yang tidak ada. Perbaiki dengan menambahkan:

- `set -e` di baris kedua
- Pengecekan `-d "$DIREKTORI"` sebelum memanggil `du`
- Pesan error yang informatif jika direktori tidak ditemukan

Uji dengan direktori yang tidak ada.

1.7 Best Practices

Script Bash yang baik mengikuti prinsip berikut:

1. Selalu tulis shebang (`#!/bin/bash`).
2. Gunakan **set -euo pipefail** di awal script.
3. Kutip semua variabel: `"$NAMA"`, bukan `$NAMA` mencegah masalah jika nilai mengandung spasi.
4. Gunakan **local** di dalam fungsi untuk mencegah variabel bocor ke global.
5. Validasi semua input sebelum diproses.
6. Nama variabel deskriptif: `DIREKTORI_BACKUP` lebih jelas dari `D`.
7. Gunakan **trap** untuk cleanup saat script berakhir.
8. Pisahkan konfigurasi dari logika: taruh variabel konfigurasi di bagian atas script.

Template script profesional yang menerapkan semua prinsip di atas:

```
#!/bin/bash
# =====
# Script: nama-script.sh
# Deskripsi: Penjelasan singkat fungsi script ini
# Penggunaan: ./nama-script.sh [-v] [-h] <argumen>
# =====

set -euo pipefail

SCRIPT_NAME=$(basename "$0")
VERBOSE=false

usage() {
    echo "Penggunaan: $SCRIPT_NAME [-v] [-h] <argumen>"
    echo "  -v  Verbose mode"
    echo "  -h  Tampilkan bantuan"
    exit 0
}

log()          { echo "[${SCRIPT_NAME}] $*"; }
log_verbose() { [ "$VERBOSE" = true ] && log "$*"; }
error_exit()  { echo "ERROR: $*" >&2; exit 1; }
cleanup()     { log_verbose "Cleanup selesai."; }
```

```
trap cleanup EXIT

while getopts "vh" OPSI; do
    case $OPSI in
        v) VERBOSE=true ;;
        h) usage ;;
        *) error_exit "Opsi tidak dikenal. Gunakan -h." ;;
    esac
done
shift $((OPTIND - 1))

[ $# -lt 1 ] && error_exit "Argumen tidak lengkap. Gunakan
-h."

log "Script dimulai"
log_verbose "Argumen: $*"

# Logika utama di sini

log "Selesai."
```

Kode 1.33: Template script Bash profesional

1.8 Tugas Praktikum

Peringatan

Kerjakan tugas di `$HOME/praktikum-os/week09/`. Script di `scripts/`, output di `logs/`.

Tugas 1 Script Absensi Kelas

Konteks: instruktur mencatat kehadiran mahasiswa dari command line.

Instruksi:

1. Buat script `absensi.sh` yang:
 - Menerima argumen nama mahasiswa dan status (`hadir/izin/alpha`)

- Menyimpan entri ke `absensi-YYYY-MM-DD.txt` dengan format `[HH:MM] NAMA - STATUS`
 - Opsi `-r`: tampilkan rekapitulasi (jumlah per status)
 - Opsi `-h`: tampilkan bantuan
2. Rekam minimal 5 entri dan tampilkan rekapitulasinya.

Konsep wajib: variabel, parameter posisional, `getopts`, `if`, `for`, fungsi, dan redirection ke file.

Tugas 2 Script Health Check Sistem

Konteks: administrator membuat pemeriksaan kondisi server sebelum maintenance.

Instruksi:

1. Buat script `healthcheck.sh` menggunakan template profesional dari bagian Best Practices.
2. Script menampilkan: tanggal/waktu, hostname, uptime, penggunaan CPU, memori, dan disk untuk setiap filesystem yang terpasang.
3. Jika penggunaan disk mana pun melebihi 80%, tampilkan peringatan.
4. Simpan hasil ke `healthcheck-YYYY-MM-DD.log` dan tampilkan ke terminal sekaligus menggunakan `tee`.
5. Opsi `-t <persen>` mengubah batas peringatan disk (default 80).

Konsep wajib: `set -euo pipefail`, `trap`, `getopts`, fungsi dengan `local`, `for`, `if`, dan `tee`.

1.9 Rangkuman

- Shebang (`#!/bin/bash`) menentukan interpreter. Tanpa shebang, perilaku script tidak terdefinisi.
- Variabel Bash tidak bertipe; aritmetika pakai `$(( ))`; output perintah ditangkap dengan `$()`.
- Parameter posisional (`$1`, `$#`, `$@`) memungkinkan script menerima argumen dari command line.
- Struktur kontrol `if`, `for`, `while`, dan `case` setara dengan Java, hanya sintaknya berbeda.

- Fungsi menerima argumen via `$1/$2`; gunakan `local` untuk variabel lokal dan `echo` untuk mengembalikan nilai.
- `getopts` memproses opsi `-x` seperti perintah Unix standar.
- `set -euo pipefail` dan `trap` adalah fondasi script yang andal dan mudah di-debug.

1.10 Referensi

1. GNU Bash Reference Manual <https://www.gnu.org/software/bash/manual/>
2. William Shotts, *The Linux Command Line*, 2nd Edition. No Starch Press.
3. Arnold Robbins dan Nelson H.F. Beebe, *Classic Shell Scripting*. O'Reilly.
4. Richard Blum dan Christine Bresnahan, *Linux Command Line and Shell Scripting Bible*, 4th Edition.
5. ShellCheck alat analisis statis untuk script Bash: <https://www.shellcheck.net>